

Deteksi Kantuk Pengemudi Pada Citra *Dashboard* Menggunakan Metode *Deep Learning Convolutional Neural Networks* Dan *Transfer Learning*

Andika Dibya Azzumardi¹ dan Kartika Fithriasari²

Departemen Statistika, Fakultas Sains dan Analitika Data, Institut Teknologi Sepuluh Nopember

Jl. Arief Rahman Hakim, Surabaya 60111 Indonesia

e-mail: 5003211116@student.its.ac.id¹ dan kartika_f@statistika.its.ac.id²

Abstrak— Kecelakaan akibat kantuk pengemudi merupakan permasalahan serius yang mendorong perlunya pengembangan sistem deteksi dini. Penelitian ini bertujuan untuk mengembangkan sistem deteksi kantuk pengemudi berbasis citra *dashboard* kendaraan dengan memanfaatkan teknologi *Deep Learning*, khususnya *Convolutional Neural Networks* (CNN) serta pendekatan *transfer learning*. Metode yang digunakan mengintegrasikan *Haar Cascade Classifier* sebagai alat deteksi fitur wajah dan mata dengan CNN untuk mengklasifikasikan kondisi mata pengemudi. Tiga arsitektur model dievaluasi dalam penelitian ini, yakni model *transfer learning* dengan arsitektur *MobileNet* dan *ResNet50*, serta model *Deep CNN* yang dibangun secara khusus (*custom*). Sebanyak 2700 data citra yang terbagi secara seimbang antara kelas mata tertutup dan terbuka digunakan untuk melatih ketiga model tersebut, dengan optimasi menggunakan teknik *hyperparameter tuning* otomatis berbasis *Optuna*. Hasil pelatihan pada dataset citra menunjukkan bahwa model *Deep CNN* yang dibangun khusus mampu mencapai akurasi validasi hingga 0,9901 dengan nilai *loss* sebesar 0,0029, yang merupakan hasil terbaik di antara ketiga model. Ketiga model yang telah dilatih tersebut selanjutnya disimpan dan diintegrasikan dengan model *Haar Cascade Classifier* untuk diuji menggunakan dataset berisi 128 video yang terbagi secara seimbang ke dalam kelas mengantuk dan tidak mengantuk. Pada tahap pengujian, model kustom *Deep CNN* memperoleh akurasi sebesar 97% dengan hanya empat kesalahan prediksi sehingga menjadikannya sebagai model terbaik. Temuan ini menegaskan potensi besar model kustom *Deep CNN* untuk diterapkan dalam sistem deteksi kantuk secara *real-time*. Namun, masih terdapat tantangan seperti fluktuasi performa selama proses pelatihan dan adanya *delay* pada implementasi *real-time*, yang perlu dikaji lebih lanjut dalam penelitian selanjutnya.

Kata Kunci— *Convolutional Neural Networks*, *Deep Learning*, *Haar Cascade Classifier*, Kantuk, *Transfer Learning*.

I. PENDAHULUAN

Keselamatan berkendara menjadi isu krusial di era modern, terutama di Indonesia di mana angka kecelakaan lalu lintas masih tinggi. Salah satu faktor utama penyebab kecelakaan, khususnya di jalan tol, adalah kondisi kelelahan dan kantuk pada pengemudi. Kondisi mengantuk secara drastis dapat menurunkan kemampuan refleks, kesadaran, dan perhatian pengemudi, sehingga upaya untuk memantau dan mendeteksi kantuk secara dini menjadi sangat penting untuk menciptakan lingkungan berkendara yang lebih aman.

Untuk mengatasi masalah ini, penelitian ini mengusulkan sistem deteksi kantuk pengemudi berbasis analisis wajah.

Pendekatan awal menggunakan algoritma *Viola-Jones* atau metode *Haar Cascade Classifier* untuk mengidentifikasi dan melacak fitur-fitur wajah secara efisien seperti mata. Metode ini dipilih karena kemampuannya yang efektif dalam mendeteksi objek wajah bahkan dalam kondisi pencahayaan yang kurang optimal dan tidak memerlukan sumber daya komputasi yang besar sehingga cocok untuk implementasi praktis [1].

Informasi fitur wajah yang diekstraksi, terutama kondisi mata yang terbuka atau tertutup, kemudian dianalisis lebih lanjut menggunakan teknologi *machine learning*, yaitu *Convolutional Neural Networks* (CNN) yang merupakan sebuah cabang dari *deep learning* yang sangat andal dalam memproses dan mengenali pola dari data gambar [2]. Kemampuan CNN untuk secara otomatis belajar fitur-fitur relevan dari citra membuatnya sangat cocok untuk mengidentifikasi pola kantuk tanpa perlu rekayasa fitur manual.

Penelitian ini tidak hanya menggunakan model CNN dasar, tetapi juga memanfaatkan dua arsitektur canggih *transfer learning*, yaitu *MobileNet* dan *ResNet50*. Teknik *transfer learning* memungkinkan model untuk menggunakan pengetahuan yang telah dipelajari dari dataset *ImageNet*, sehingga mempercepat proses pelatihan dan meningkatkan kemampuan deteksi pola visual yang kompleks [3].

MobileNet dipilih karena efisiensinya yang tinggi dan kebutuhan sumber daya komputasi yang rendah [4]. Dengan teknik *depthwise separable convolution*, *MobileNet* dapat mendeteksi indikator kantuk seperti mata tertutup dengan cepat tanpa mengorbankan akurasi secara signifikan [5]. Sementara itu, *ResNet50* menawarkan pendekatan yang berbeda dengan arsitektur yang sangat dalam berkat penggunaan blok residual. Mekanisme ini memungkinkan *ResNet50* untuk mengatasi masalah *vanishing gradient* sehingga mampu melatih jaringan dengan ratusan lapisan tanpa kehilangan kemampuan untuk belajar dari data [6]. Selanjutnya, ketiga model ini diuji pada *training set* dan *validation set* untuk melihat performanya melalui akurasi, *loss*, dan *confusion matrix*.

Setelah proses pelatihan, model CNN yang dihasilkan diintegrasikan dengan *Haar Cascade Classifier* untuk diaplikasikan pada *datatest* berupa rekaman video pengemudi. Performa ketiga model kembali diuji dengan metrik yang dihasilkan oleh *classification report* dan *confusion matrix*. Setelah itu, dilakukan pemilihan model terbaik, baik saat *training* maupun *testing*. Model terbaik dari penelitian ini selanjutnya diimplementasikan pada *website*, di mana

pengguna bisa melakukan deteksi kantung secara *realtime* maupun dengan mengunggah video.

II. TINJAUAN PUSTAKA

A. Convolutional Neural Networks

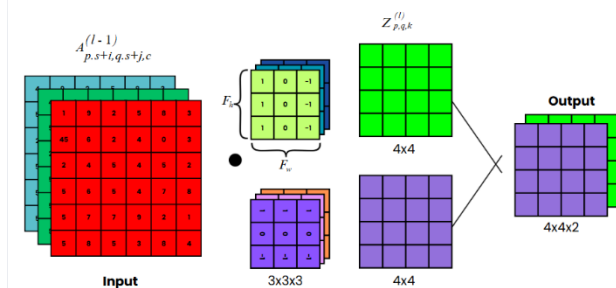
Lapisan konvolusi (*convolutional layer*) adalah komponen fundamental dalam *neural networks* yang dirancang khusus untuk memproses data spasial seperti citra digital [7]. Inti dari lapisan ini adalah operasi konvolusi, yaitu proses mengaplikasikan sebuah *kernel* $W_{i,j,c,k}^{(l)}$ secara bergeser ke seluruh bagian citra input. Pada setiap pergeseran, dilakukan operasi *dot product* antara nilai-nilai pada *kernel* dengan nilai-nilai piksel citra $A_{p,s+i,q,s+j,c}^{(l-1)}$ yang merupakan *output* dari nilai sebelumnya. Proses ini menghasilkan sebuah matriks baru pra-aktivasi $Z_{p,q,k}^{(l)}$, yang merepresentasikan hasil ekstraksi fitur-fitur tertentu dari citra asli, seperti deteksi tepi, tekstur, atau bentuk.

$$Z_{p,q,k}^{(l)} = \left(\sum_{c=1}^{C_{in}} \sum_{i=1}^{F_h} \sum_{j=1}^{F_w} W_{i,j,c,k}^{(l)} \cdot A_{p,s+i,q,s+j,c}^{(l-1)} + b_k^{(l)} \right) \quad (1)$$

Setelah itu, dilakukan aktivasi nilai $Z_{p,q,k}^{(l)}$ dengan fungsi aktivasi tertentu seperti pada Persamaan 2.

$$A_{p,q,k}^{(l)} = f(Z_{p,q,k}^{(l)}) \quad (2)$$

Perhitungan nilai pada Persamaan 1 dan 2 diilustrasikan pada Gambar 1 berikut.



Gambar 1. Proses Konvolusi

Pada CNN terdapat *pooling layer* yang merupakan operasi *downsampling* yang esensial, bertujuan untuk mengurangi dimensi spasial (*feature map*) agar dapat menyederhanakan kompleksitas jaringan dan membuat komputasi lebih efisien. Terdapat beberapa operasi *pooling*, seperti *Max Pooling* yang memilih nilai maksimum dari sebuah area untuk menangkap fitur yang paling dominan seperti pada Persamaan 3.

$$P_{p,q,k}^{(l)} = \max_{i=1..l_h, j=1..l_w} (A_{p,s+i,q,s+j,k}^{(l-1)}) \quad (3)$$

Sementara itu, *Average Pooling* menghitung nilai rata-rata untuk mendapatkan representasi fitur yang lebih umum seperti pada Persamaan 4.

$$P_{p,q,k}^{(l)} = \frac{1}{l_h \cdot l_w} \sum_{i=0}^{l_h-1} \sum_{j=0}^{l_w-1} A_{p,s+i,q,s+j,k}^{(l-1)} \quad (4)$$

Selain itu, ada juga *Global Average Pooling* yang secara drastis mereduksi setiap *feature map* menjadi satu nilai tunggal, yang

sangat efektif dalam mengurangi jumlah parameter sebelum data dimasukkan ke lapisan *fully connected*.

Setelah itu, sebelum dimasukkan ke *fully connected layer*, *feature map* dua dimensi tersebut diubah menjadi *array* satu dimensi atau vektor yang melalui proses *flattening* yang bisa dihasilkan melalui Persamaan 5 berikut.

$$a_{flat}^{(l)} = \text{flatten}(P^{(l)}) \quad (5)$$

Ada 2 *activation function* yang digunakan pada penelitian ini. Fungsi aktivasi *ReLU* digunakan karena kesederhanaan dan keefektifannya dalam mempercepat proses pelatihan dengan mencegah saturasi neuron [8]. Persamaan 8 menjelaskan fungsi aktivasi ini yang menjadikan nilai minimum *output*nya 0.

$$f_{ReLU}(z) = \max(0, z) \quad (6)$$

Fungsi aktivasi *Softmax* yang juga digunakan pada penelitian ini dengan *output* 2 kelas seperti pada Persamaan 7.

$$\hat{y}_c = f_{Softmax}(z)_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}} \quad (7)$$

Fungsi *softmax* memberikan estimasi probabilitas untuk setiap kelas, dengan kelas yang menjadi target memiliki probabilitas tertinggi [9]. Ketika menghitung nilai *softmax* pada *output* vektor, fungsi ini tidak diterapkan secara independen pada setiap elemen, melainkan diterapkan secara bersamaan pada seluruh *output* karena *softmax* mendistribusikan probabilitas secara proporsional ke setiap node *output* [10].

Untuk mengukur seberapa baik model yang sedang dilatih dalam menghasilkan prediksi yang tepat digunakanlah *loss function* seperti pada persamaan 8.

$$L(y_i, \hat{y}_i) = J(\theta) = -\frac{1}{N} \times \sum_{i=1}^N \sum_{c=1}^C (y_{ic} \log \hat{y}_{ic}) \quad (8)$$

Dalam *deep learning*, optimasi diperlukan untuk menghitung *loss function* yang dapat didiferensiasi dan bersifat kontinu dengan menggunakan teknik berbasis gradien yang memanfaatkan informasi dari orde pertama atau orde kedua untuk memperbarui parameter pembobot dan mencari titik optimal [11]. Nilai gradien adalah turunan dari *loss function* terhadap parameter seperti pada Persamaan 9.

$$g^{(t)} = \nabla J(\theta)^{(t)} = \frac{\partial J(\theta)^{(t)}}{\partial \theta^{(t)}} \quad (9)$$

Hasil penurunan tersebut menghasilkan Persamaan 10.

$$\frac{\partial J(\theta)^{(t)}}{\partial \theta^{(t)}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) x_i^{(t)} \quad (10)$$

Selanjutnya, dilakukan pembaruan estimasi momen pertama dan kedua seperti pada Persamaan 11 dan 12.

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1) g^{(t)} \quad (11)$$

$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2) (g^{(t)})^2 \quad (12)$$

Koreksi biasnya terdefinisi pada Persamaan 13 dan 14 berikut.

$$\widehat{m}^{(t)} = \frac{m^{(t)}}{1 - \beta_1^t} \quad (13)$$

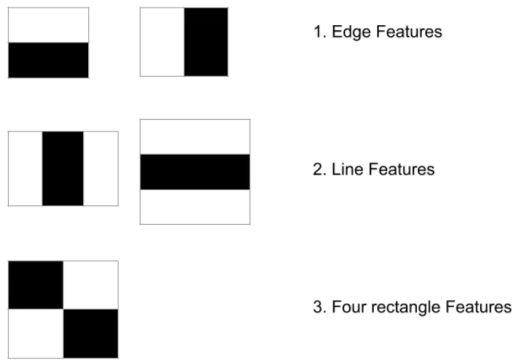
$$\widehat{v}^{(t)} = \frac{v^{(t)}}{1 - \beta_2^t} \quad (14)$$

Didapatkanlah hasil akhir pembaruan parameter pada Persamaan 15 berikut.

$$\theta_t = \theta^{(t-1)} - \alpha \frac{\widehat{m^{(t)}}}{\sqrt{v^{(t)}} + \epsilon} \quad (15)$$

B. Haar Cascade Classifier

Algoritma *Viola-Jones*, atau yang lebih dikenal sebagai *Haar Cascade*, adalah sebuah metode deteksi objek yang dirancang untuk bekerja secara *real-time* dan efisien, khususnya untuk deteksi wajah. Konsep intinya adalah dengan menggunakan ribuan fitur sederhana yang disebut *Haar-like features*, yaitu perhitungan selisih intensitas piksel pada area-area persegi panjang untuk mendeteksi pola dasar seperti garis dan tepi seperti pada Gambar 2 [12]. Untuk mengatasi masalah komputasi yang sangat tinggi dalam menghitung semua fitur ini, algoritma ini memanfaatkan teknik *Integral Image* yang memungkinkan kalkulasi fitur dilakukan dengan sangat cepat. Setelah itu, dari puluhan ribu fitur yang ada, algoritma *machine learning* bernama *AdaBoost* digunakan untuk secara otomatis memilih sekelompok kecil fitur yang paling relevan dan menggabungkannya menjadi sebuah *strong classifier*.



Gambar 2. *Haar-like Features*

Untuk mencapai kecepatan deteksi yang tinggi, fitur-fitur yang telah dipilih oleh *AdaBoost* tidak diaplikasikan sekaligus, melainkan disusun dalam sebuah struktur bernama *Cascade Classifier*. Klasifikator ini terdiri dari beberapa tahapan, dimulai dari yang paling sederhana hingga yang paling kompleks. Setiap area pada gambar akan dievaluasi oleh tahapan pertama yang menggunakan sangat sedikit fitur untuk secara cepat menyingkirkan area yang jelas-jelas bukan objek (misalnya, dinding kosong). Hanya area yang lolos dari tahapan awal yang akan diproses oleh tahapan selanjutnya yang lebih rumit. Pendekatan bertingkat ini memastikan bahwa sebagian besar waktu komputasi hanya difokuskan pada area yang paling potensial, sehingga memungkinkan deteksi objek yang akurat dan cepat bahkan pada video secara langsung.

C. Evaluasi Hasil Klasifikasi

Untuk menilai performa model CNN, evaluasi dilakukan dengan menganalisis hasil prediksi melalui *confusion matrix*. Matriks ini secara rinci mengkategorikan prediksi ke dalam empat komponen, yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Dari komponen-komponen ini, dihitung beberapa metrik penting pada Persamaan 16 dan 17 seperti *True Positive Rate* (TPR) yang mengukur kemampuan model dalam mengenali kelas

positif, dan *False Positive Rate* (FPR) untuk melihat tingkat kesalahan dalam memprediksi kelas negatif [13].

$$TPR = \frac{TP}{FN + TP} \quad (16)$$

$$FPR = \frac{FP}{TN + FP} \quad (17)$$

Selain itu, metrik akurasi digunakan untuk mengukur proporsi total prediksi yang benar terhadap keseluruhan data.

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (18)$$

Persamaan 18 dapat memberikan gambaran umum mengenai ketepatan model secara keseluruhan.

III. METODOLOGI PENELITIAN

A. Sumber Data

Data yang digunakan pada penelitian ini terdiri dari 2 jenis, yaitu dataset citra mata terbuka dan tertutup yang diambil dari *MRL Eye Dataset* dan *Drowsiness Dataset* serta dataset video rekaman pengemudi yang diambil melalui situs *istockphoto*.

B. Variabel Penelitian

Terdapat dua variabel pada penelitian ini, yaitu variabel X sebagai nilai dari ekstraksi warna pada tiap piksel dan variabel Y yang menggambarkan label citra mata seperti pada Tabel 1.

Tabel 1.

Variabel Penelitian				
No.	Variabel	Keterangan	Skala	Nilai
1	X	Nilai dari ekstraksi warna tiap pikselnya	Rasio	0-255
2	Y	Label citra mata	Nominal	[1,0] : Mata tertutup [0,1] : Mata terbuka

C. Struktur Data

Struktur data pada penelitian ini dapat dilihat pada Tabel 2.

Tabel 2.

Struktur Data

Piksel Citra (X)		Label (Y)
$(X_{1,1,1,R}, X_{1,1,1,G}, X_{1,1,1,B})$	$\dots$	$(X_{1,1,32,R}, X_{1,1,32,G}, X_{1,1,32,B})$
$\vdots$	$\ddots$	$\vdots$
$(X_{1,32,1,R}, X_{1,32,1,G}, X_{1,32,1,B})$	$\dots$	$(X_{1,32,32,R}, X_{1,32,32,G}, X_{1,32,32,B})$
$\vdots$	$\ddots$	$\vdots$
$(X_{2,1,1,R}, X_{2,1,1,G}, X_{2,1,1,B})$	$\dots$	$(X_{2,1,32,R}, X_{2,1,32,G}, X_{2,1,32,B})$
$\vdots$	$\ddots$	$\vdots$
$(X_{2,32,1,R}, X_{2,32,1,G}, X_{2,32,1,B})$	$\dots$	$(X_{2,32,32,R}, X_{2,32,32,G}, X_{2,32,32,B})$
$\vdots$	$\ddots$	$\vdots$
$(X_{N,1,1,R}, X_{N,1,1,G}, X_{N,1,1,B})$	$\dots$	$(X_{N,1,32,R}, X_{N,1,32,G}, X_{N,1,32,B})$
$\vdots$	$\ddots$	$\vdots$
$(X_{N,32,1,R}, X_{N,32,1,G}, X_{N,32,1,B})$	$\dots$	$(X_{N,32,32,R}, X_{N,32,32,G}, X_{N,32,32,B})$

D. Langkah Analisis

Berikut merupakan langkah – langkah analisis untuk melakukan penelitian ini.

1. Mengumpulkan dan memproses dataset citra mata MRL

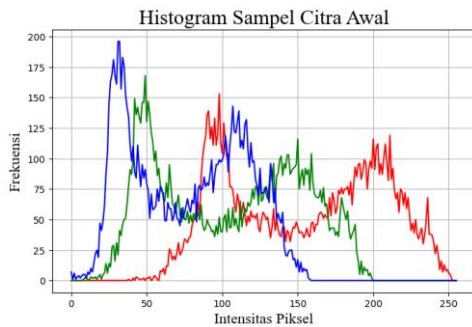
Eye melalui augmentasi dan normalisasi.

2. Membangun tiga model CNN (*Deep CNN*, *MobileNet*, *ResNet50*) dan mengoptimalkan *hyperparameter*-nya menggunakan *Optuna*.
3. Mengevaluasi performa awal ketiga model pada data citra dan menyimpan arsitektur terbaiknya.
4. Menerapkan *Haar Cascade Classifier* untuk mendeteksi dan mengekstraksi area mata dari data video pengemudi.
5. Mengklasifikasikan kondisi mata pada setiap *frame* video menggunakan model yang telah dilatih untuk menentukan status kantuk.
6. Melakukan pelabelan manual pada data video sebagai pembandingan (*ground truth*) untuk hasil prediksi model.
7. Menganalisis kinerja akhir setiap model pada data video dengan menghitung *confusion matrix* dan metrik terkait.
8. Membandingkan performa antar model, menarik kesimpulan, dan merumuskan saran untuk pengembangan di masa depan.

IV. HASIL DAN PEMBAHASAN

A. Preprocessing Data

Untuk memodelkan jaringan saraf pada MRL *Eye Dataset*, langkah pertama yang perlu dilakukan adalah *preprocessing* gambar-gambar tersebut. Gambar-gambar dalam dataset ini memiliki ukuran yang berbeda-beda, sehingga perlu dilakukan *resizing* agar ukuran inputnya seragam. Semua gambar diubah menjadi ukuran 32 x 32. Hal ini bertujuan agar model tetap ringan dan dapat digunakan pada berbagai perangkat dengan kapasitas yang berbeda.

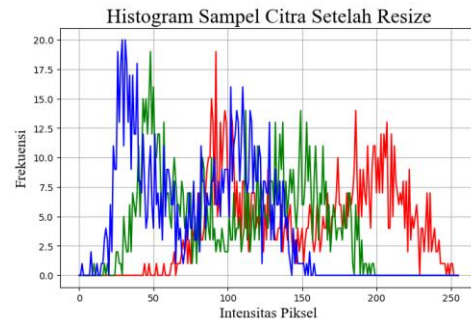


Gambar 3. Histogram Sampel Citra Awal

Gambar 3 merupakan histogram awal salah satu sampel citra yang digunakan pada dataset yang memiliki ukuran asli 103x111. Struktur data pada sampel gambar awal tersebut masih memiliki dimensi asli seperti yang terlihat pada matriks berikut.

$$\mathbf{X}_{awal} = \begin{pmatrix} (221, 165, 128) & \cdots & (180, 137, 95) \\ \vdots & \ddots & \vdots \\ (232, 174, 134) & \cdots & (231, 180, 133) \end{pmatrix}$$

Setelah melalui proses *resizing* menjadi 32x32, frekuensi nilai piksel dari masing-masing *channel* warna pada hisotogramnya menjadi lebih sempit dan kecil seperti pada Gambar 4 sebagai berikut.



Gambar 4. Histogram Sampel Citra Setelah *Resize*

$$\mathbf{X}_{resize} = \begin{pmatrix} (220, 164, 128) & \cdots & (185, 142, 99) \\ \vdots & \ddots & \vdots \\ (235, 177, 137) & \cdots & (234, 183, 136) \end{pmatrix}$$

B. Evaluasi Peforma Model Pada MRL Eye Dataset

Sebelum melakukan *training* pada MRL *Eye Dataset*, dilakukan konfigurasi 4 *hyperparameter* yang berbeda untuk mengoptimalkan akurasi model seperti pada Tabel 3.

Tabel 3.

Konfigurasi *Hyperparameter*

<i>Hyperparameter</i>	Konfigurasi
<i>Learning Rate</i>	[1e-5, 1e-4, 1e-3, 1e-2, 1e-1]
<i>Dropout</i>	[0,05; 0,1; 0,2; 0,3; 0,4]
<i>Batch Size</i>	[32, 64, 128]
Jumlah <i>Epoch</i>	[20, 30, 40, 50]

1. Peforma Model *MobileNet*

Arsitektur *MobileNet pre-trained* secara inheren dirancang untuk tugas klasifikasi dengan jumlah kategori output yang berbeda dari kebutuhan analisis dalam penelitian ini, maka lapisan klasifikasi (*fully connected layer*) bawaan dari model *MobileNet* tidak diikutsertakan. Hal ini dicapai dengan mengatur parameter *include_top=False* saat memuat model. Detail arsitektur *MobileNet* dapat dilihat pada Tabel 4.

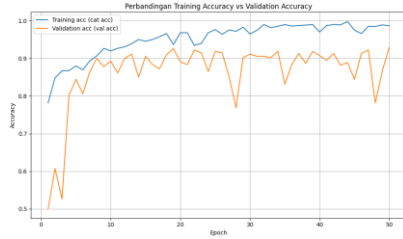
Tabel 4.

Arsitektur Model *MobileNet*

<i>Layer (type)</i>	<i>Output Shape</i>	<i>Parameter</i>
input_12 (InputLayer)	[(None, 32, 32, 3)]	0
conv1 (Conv2D)	(None, 16, 16, 32)	864
conv1_bn (BatchNormalization)	(None, 16, 16, 32)	128
conv1_relu (ReLU)	(None, 16, 16, 32)	0
conv_dw_1 (DepthwiseConv2D)	(None, 16, 16, 32)	288
conv_dw_1_bn (BatchNormalization)	(None, 16, 16, 32)	128
⋮	⋮	⋮
conv_pw_13_relu (ReLU)	(None, 1, 1, 1024)	0
global_average_pooling2d	(None, 1024)	0
dropout_13 (Dropout)	(None, 512)	0
dense_64 (Dense)	(None, 2)	2050
Total Parameter		3.230.914

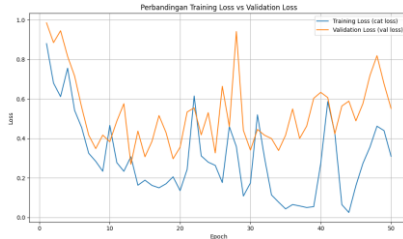
Setelah itu, dilakukanlah proses *training* model dengan berbagai kombinasi model dengan berbagai kombinasi *hyperparameter* yang telah ditetapkan sebelumnya. Hasilnya, dari 30 percobaan, didapatkan kombinasi *hyperparameter*

terbaik adalah dengan 40 *epoch*, *batch size* 64, *dropour rate* 0,2, dan *learning rate* 0,01 pada percobaan 19.



Gambar 5. Perbandingan Akurasi Model *MobileNet*

Gambar 5 merupakan perbandingan akurasi pada *training set* dan *validation set* percobaan 19. Model *MobileNet* mencapai akurasi di *validation set* terbaik di angka 0.9296. Tetapi, pada beberapa *epoch*, akurasinya masih cukup fluktuatif.



Gambar 6. Perbandingan Loss Model *MobileNet*

Gambar 6 di atas merupakan perbandingan *loss* pada *training set* dan *validation set* percobaan 19. Gambar tersebut memperlihatkan hasil yang cukup fluktuatif baik di *data training* dan *data validation* sepanjang 50 *epoch*. Kedua gambar di atas sudah cukup menjelaskan bahwa model *MobileNet* tidak terlalu yakin dalam melakukan klasifikasi sehingga banyak terjadi fluktuasi yang curam di banyak *epoch*.

2. Peforma Model ResNet50

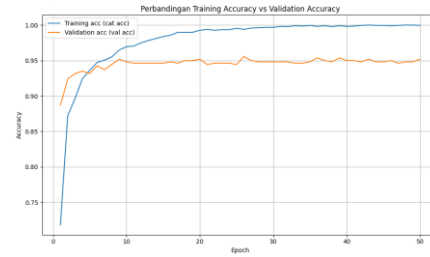
Arsitektur *ResNet50* yang lebih dalam memiliki parameter total sebanyak 23.620.530 dirancang untuk mengolah citra berukuran 32x32 piksel dengan tiga saluran warna. Model ini diawali dengan layer input yang kemudian diteruskan ke tahap *padding* nol untuk memperbesar dimensi spasial sebelum memasuki layer konvolusi pertama yang menggunakan 64 filter berukuran kernel tertentu untuk menghasilkan fitur dasar sebanyak 64 *channel* dengan dimensi 16x16. Detail mengenai arsitektur ini dapat dilihat pada Tabel 5.

Tabel 5

Arsitektur Model *ResNet50*

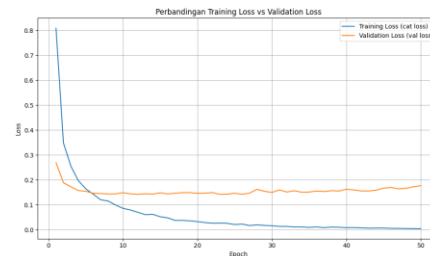
Layer (type)	Output Shape	Parameter
input_5 (InputLayer)	[(None, 32, 32, 3)]	0
conv1_pad (ZeroPadding2D)	(None, 38, 38, 3)	0
conv1_conv (Conv2D)	(None, 16, 16, 64)	9472
conv1_bn (BatchNormalization)	(None, 16, 16, 64)	256
conv1_relu (Activation)	(None, 16, 16, 64)	0
⋮	⋮	⋮
Flatten	(None, 2048)	23587712
dense_22 (Dense)	(None, 16)	32784
dropout_5 (Dropout)	(None, 16)	0
dense_23 (Dense)	(None, 2)	34
Total parameter		23.620.530

Setelah dilakukan percobaan sebanyak 30 kali dengan kombinasi *hyperparameter* yang berbeda, percobaan terbaik pada model *ResNet50* didapati menggunakan kombinasi *hyperparameter* *epoch* 40, *batch size* 32, *dropout rate* 0,3, dan *learning rate* 0,1.



Gambar 7. Perbandingan Akurasi Model *ResNet50*

Dari Gambar 7, terlihat bahwa model *ResNet50* stabil di angka sekitar 95 persen dan mencapai performa terbaiknya di angka 0,9556. Dari *epoch* 10 tidak terjadi adanya perubahan nilai akurasi yang signifikan sampai akhir.



Gambar 8. Perbandingan Loss Model *ResNet50*

Dari Gambar 8, terlihat juga *loss* yang stagnan mulai dari *epoch* awal sampai akhir. Walaupun hasil nilai akurasi dan *loss* yang cukup baik, perlu pengkajian ulang alasan model yang cenderung sangat stabil pada akurasi dan *loss* pada data validasi.

3. Peforma Model Deep CNN

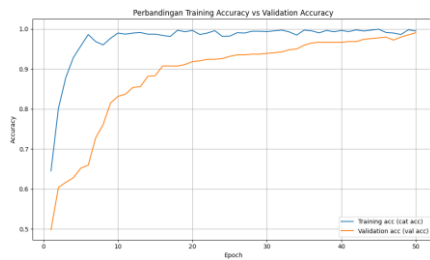
Model *Deep CNN* merupakan model yang dibuat dari awal yang terinspirasi dari beberapa arsitektur seperti *ResNet* dan *SENet*. Namun, model ini memiliki jumlah parameter yang tidak sebesar *ResNet* sehingga bisa menjadi model alternatif yang tidak terlalu berat dengan kualitas yang baik.

Tabel 6.

Arsitektur Model *Deep CNN*

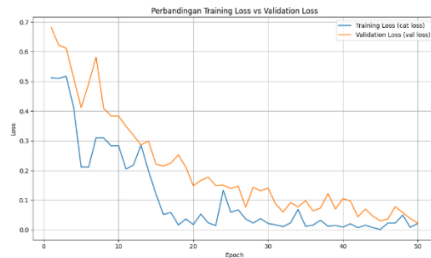
Layer (type)	Output Shape	Parameter
input_5 (InputLayer)	[(None, 32, 32, 3)]	0
conv2d_102 (Conv2D)	(None, 32, 32, 64)	1792
batch_normalization_90 (BatchNormalization)	(None, 32, 32, 64)	256
conv2d_103 (Conv2D)	(None, 32, 32, 64)	4160
batch_normalization_91 (BatchNormalization)	(None, 32, 32, 64)	256
⋮	⋮	⋮
dropout_4 (Dropout)	(None, 512)	0
dense_79 (Dense)	(None, 256)	131328
dropout_5 (Dropout)	(None, 256)	0
dense_80 (Dense)	(None, 2)	514
Total params:		10.453.618

Hasil pelatihan model ini dapat dilihat pada gambar di bawah ini.



Gambar 9. Perbandingan Akurasi Model Deep CNN

Gambar 9 di atas menunjukkan akurasi model Deep CNN yang sangat baik. Nilai akurasi cenderung terus mengalami kenaikan dari awal *epoch* sampai akhir hingga mencapai puncaknya pada angka 0,9907. Hasil yang hampir sempurna tersebut merupakan nilai yang terbaik di antara ketiga model CNN yang digunakan.



Gambar 10. Perbandingan Loss Model Deep CNN

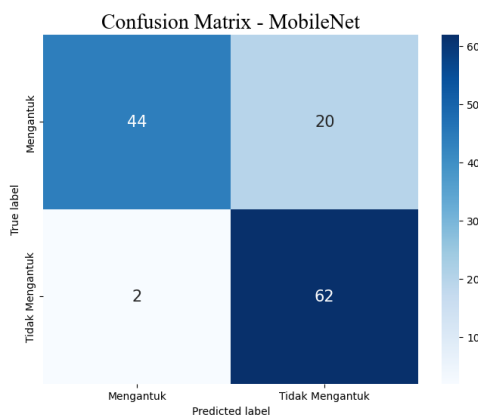
Gambar 10 di atas menunjukkan nilai *loss* pada percobaan terbaik model Deep CNN yang cenderung selalu menurun pada setiap *epoch*-nya. Berbeda dengan nilai akurasinya, nilai *loss* model ini beberapa kali mengalami fluktuasi walaupun akhirnya tetap menurun dan menghasilkan nilai *loss* terendah di antara semua model.

C. Evaluasi Peforma Model di Datatest Video

Setelah dilakukan *training* pada ketiga model pada dataset citra mata, dilakukanlah implementasi ketiga model pada *datatest* berisi video orang mengemudi.

1. Peforma Model MobileNet

Pada model MobileNet, hasil *confusion matrix* menunjukkan cukup banyak kesalahan pada kelas mengantuk. Namun, pada kelas tidak mengantuk, hanya ada 2 kesalahan prediksi.



Gambar 11. Confusion Matrix Model MobileNet di Datatest

Untuk melihat peforma keseluruhan pada model ini, digunakanlah *classification report* dengan berbagai metrik yang bisa dilihat pada Tabel 7.

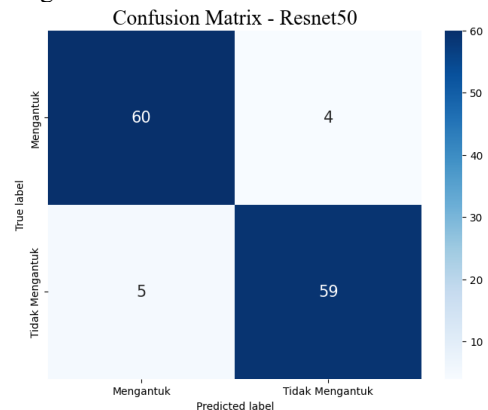
Tabel 7

Classification Report Model MobileNet di Datatest					
Kelas	Precision	Recall	F1-Score	Support	Akurasi
Tidak Mengantuk	0,96	0,97	0,85	64	0,83
Mengantuk	0,76	0,69	0,80	64	

Dari Tabel 7, dapat dilihat bahwa pada kelas mengantuk model memiliki *precision* 0,96, *recall* 0,69, dan *F1-Score* 0,80 yang menunjukkan bahwa model sangat akurat dalam memprediksi kelas mengantuk walaupun memiliki *recall* yang lebih rendah. Oleh karena itu, ada beberapa data untuk kelas mengantuk yang tidak terdeteksi dengan baik. Dengan akurasi keseluruhan 0,83, model ini menunjukkan kinerja yang baik meskipun masih ada kesalahan dalam mendeteksi kedua kelas, terutama pada kelas mengantuk.

2. Peforma Model ResNet50

Gambar 12 menjelaskan peforma model ResNet50 pada *datatest*. Terdapat total 9 jumlah kesalahan prediksi dari total 128 data. Nilai tersebut merupakan hasil yang sudah cukup baik dan berimbang di antara kedua kelas.



Gambar 12. Confusion Matrix Model ResNet50 di Datatest

Selain itu, keseluruhan metrik evaluasi pada model ini dapat dilihat pada Tabel 8 berikut. Tabel tersebut menjelaskan peforma model ResNet50 pada *datatest* yang sudah cukup memuaskan. Dengan nilai hasil di atas 90 di semua kelas, model ini mengindikasikan peforma yang cukup baik dan bisa digunakan pada kasus riil.

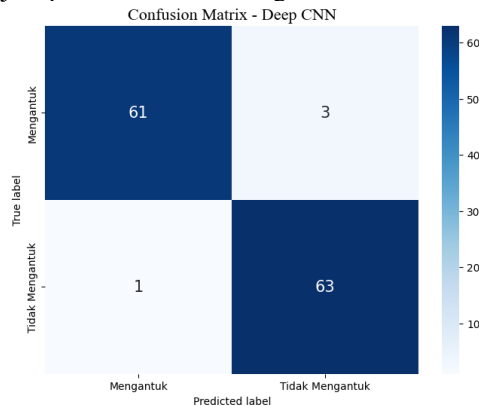
Tabel 8

Classification Report Model ResNet50 di Datatest					
Kelas	Precision	Recall	F1-Score	Support	Akurasi
Tidak Mengantuk	0,92	0,94	0,93	64	0,93
Mengantuk	0,94	0,92	0,93	64	

3. Peforma Model Deep CNN

Model Deep CNN pada saat *training* menghasilkan nilai akurasi tertinggi dan nilai *loss* terendah di antara ketiga model, kembali menunjukkan peforma maksimalnya pada *datatest*. Hal itu dibuktikan dengan hanya ada total 4 kesalahan

prediksi klasifikasi saja seperti pada Gambar 13 berikut. Sama seperti model lainnya, kesalahan pada model *Deep CNN* juga banyak terjadi pada kelas tidak mengantuk.



Gambar 13. Confusion Matrix Model Deep CNN di Datatest

Untuk melihat metrik evaluasi lainnya, dilakukanlah perhitungan *classification report* seperti pada Tabel 9 berikut.

Tabel 9.
Classification Report Model Deep CNN di Datatest

Kelas	Precision	Recall	F1-Score	Support	Akurasi
Tidak Mengantuk	0,98	0,95	0,97	64	0,97
Mengantuk	0,95	0,98	0,97	64	

Dari tabel di atas, terlihat adanya konsistensi nilai yang sangat terjaga dengan baik di semua metrik dan di kedua kelas. Nilai – nilai tersebut lagi-lagi menjadi nilai yang terbaik di antara ketiga model yang digunakan.

D. Pemilihan Model Terbaik

Dari 3 model CNN yang digunakan, selanjutnya dilakukanlah pemilihan model terbaik berdasarkan evaluasi di data citra maupun *datatest* video. Tabel 10 menunjukkan perbandingan performa ketiga model yang digunakan.

Tabel 10.
Perbandingan Keseluruhan Peforma Model

Model	Dataset Citra		Datatest Video
	Validation Accuracy	Validation Loss	Akurasi
MobileNet	0,9296	0,113	0,83
Resnet50	0,9556	0,1425	0,93
Deep CNN	0,9907	0,0229	0,97

Dari tabel di atas, terlihat bahwa model model *Deep CNN* unggul pada semua metrik, baik pada dataset citra maupun dataset video. Hal ini menunjukkan bahwa model *Deep CNN* layak digunakan lebih lanjut pada kasus di dunia nyata.

E. Evaluasi Kesalahan Prediksi

Pada *datatest* video, masih terdapat beberapa kesalahan prediksi klasifikasi yang dilakukan oleh model terbaik, yaitu model *Deep CNN*. Gambar 14 menunjukkan bahwa model tersebut gagal mendeteksi bagian mata dari sebuah objek. Hal itu diduga karena sudut kamera yang tidak secara langsung

mengarah ke wajah bagian depan sehingga model kesulitan mendeteksi area mata.



Gambar 14. Kegagalan Deteksi Objek di Kelas Mengantuk

Selain itu, pada Gambar 15, model lagi-lagi gagal mendeteksi bagian mata. Berbeda dengan gambar sebelumnya yang sama sekali tidak mendeteksi bagian mata, pada contoh ini, model salah dalam melakukan deteksi bagian mata. Area lubang hidung yang memiliki struktur warna yang mirip dengan mata membuat model salah melakukan deteksi sehingga hasil prediksinya juga salah.



Gambar 15. Kesalahan Deteksi dan Klasifikasi di Kelas Mengantuk

Kesalahan prediksi klasifikasi juga terjadi pada kelas tidak mengantuk. Hal ini dicontohkan pada Gambar 16 di mana model salah dalam melakukan klasifikasi citra mata terbuka dan tertutup.



Gambar 16. Kesalahan Klasifikasi di Kelas Tidak Mengantuk

Namun, pada video yang sama di *frame* yang berbeda, terlihat bahwa model gagal mendeteksi bagian mata secara penuh. Hal ini juga bisa mengganggu jalannya prediksi seperti pada Gambar 17.



Gambar 17. Kegagalan Deteksi Objek di Kelas Tidak Mengantuk

V. KESIMPULAN/RINGKASAN

A. Kesimpulan

Penelitian ini bertujuan untuk membandingkan tiga model berbasis CNN, yaitu *MobileNet*, *ResNet50*, dan sebuah model kustom bernama *Deep CNN* untuk klasifikasi mata terbuka dan tertutup pada MRL *Eye Dataset* serta pada data video pengemudi di dunia nyata. Setelah melalui proses *hyperparameter tuning* otomatis menggunakan *Optuna*, setiap model dievaluasi pada data citra dan diimplementasikan pada data video dengan bantuan *Haar Cascade Classifier* untuk deteksi mata. Hasilnya secara konsisten menunjukkan bahwa model *Deep CNN* yang dibangun dari awal memiliki performa paling unggul di kedua skenario pengujian, mengalahkan model *transfer learning MobileNet* dan *ResNet50*.

Meskipun unggul, model *Deep CNN* bukannya tanpa kekurangan. Pada MRL *Eye Dataset*, model ini mencapai akurasi validasi puncak 0,9901 dengan hanya 5 kesalahan prediksi, namun grafiknya menunjukkan sedikit fluktuasi selama pelatihan. Pada pengujian video, di mana model ini meraih akurasi 97%, beberapa kesalahan klasifikasi masih terjadi, umumnya disebabkan oleh sudut kamera yang tidak ideal dan pencahayaan yang buruk. Lebih lanjut, saat diimplementasikan pada sebuah *website* untuk deteksi *real-time*, ditemukan adanya sedikit *delay* atau kelambatan antara gerakan mata pengguna dan respons prediksi dari sistem, yang disebabkan oleh proses integrasi antara *frontend* dan *backend*.

B. Saran

Untuk penelitian selanjutnya, disarankan beberapa perbaikan fundamental untuk meningkatkan robustitas dan kepraktisan model. Pertama, memperkaya dan memperbanyak keragaman dataset, baik dengan mencari data baru maupun menggunakan teknik generasi gambar seperti GAN, untuk melatih model agar lebih andal dalam berbagai kondisi. Kedua, mengganti metode deteksi mata *Haar Cascade* yang memiliki keterbatasan dengan alternatif yang lebih modern dan kuat seperti *MediaPipe* atau YOLO, agar dapat mengatasi tantangan sudut kamera dan pencahayaan yang sulit. Terakhir, dari sisi implementasi, disarankan untuk mengembangkan proyek ini dari purwarupa *website* menjadi sebuah perangkat keras (*hardware*) khusus yang terintegrasi langsung di dalam kendaraan untuk penggunaan yang lebih cepat, akurat, dan praktis di dunia nyata.

DAFTAR PUSTAKA

- [1] B. Fatima, A. R. Shahid, S. Ziauddin, A. A. Safi, dan H. Ramzan, "Driver Fatigue Detection Using Viola Jones and Principal Component Analysis," *Applied Artificial Intelligence*, vol. 34, no. 6, hlm. 456–483, Mei 2020, doi: 10.1080/08839514.2020.1723875.
- [2] Dr. A. Rosebrock, *Deep Learning for Computer Vision with Python*. 2019.
- [3] A. Krizhevsky, I. Sutskever, dan G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," dalam *Advances in Neural Information Processing Systems*, F. Pereira, C. J. Burges, L. Bottou, dan K. Q. Weinberger, Ed., Curran Associates, Inc., 2012. [Daring]. Tersedia pada: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
- [4] M. Poursadeghiyan, A. Mazloumi, G. N. Saraji, M. M. Baneshi, A. Khammar, dan M. H. Ebrahimi, "Using Image Processing in the Proposed Drowsiness Detection System Design," 2018. [Daring]. Tersedia pada: <http://ijph.tums.ac.ir>
- [5] C. Jiang, Z. Jiang, dan D. Shi, "Analysis Model of Image Colour Data Elements Based on Deep Neural Network," *Comput Intell Neurosci*, vol. 2022, 2022, doi: 10.1155/2022/7631788.
- [6] A. Howard dkk., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," Jan 2017, doi: 10.48550/arXiv.1704.04861.
- [7] A. Rohim dan Y. Arum Sari, "Convolution Neural Network (CNN) Untuk Pengklasifikasian Citra Makanan Tradisional," 2019. [Daring]. Tersedia pada: <http://j-ptiik.ub.ac.id>
- [8] C. C. Aggarwal, *Neural Networks and Deep Learning*, 1 ed. Springer Cham, 2018.
- [9] R. FATURRAHMAN, Y. S. HARIYANI, dan S. HADIYOSO, "Klasifikasi Jajanan Tradisional Indonesia berbasis Deep Learning dan Metode Transfer Learning," *ELKOMIKA: Jurnal Teknik Energi Elektrik, Teknik Telekomunikasi, & Teknik Elektronika*, vol. 11, no. 4, hlm. 945, Okt 2023, doi: 10.26760/elkomika.v11i4.945.
- [10] E. Solovyeva dan A. Abdullah, "Binary and multiclass text classification by means of separable convolutional neural network," *Inventions*, vol. 6, no. 4, Des 2021, doi: 10.3390/inventions6040070.
- [11] S. Ruder, "An overview of gradient descent optimization algorithms," *CoRR*, vol. abs/1609.04747, 2016, [Daring]. Tersedia pada: <http://arxiv.org/abs/1609.04747>
- [12] M. Tyagi, "Viola Jones Algorithm and Haar Cascade Classifier."
- [13] C. Igiri, A. Uzoma, dan A. Silas, "Effect of Learning Rate on Artificial Neural Network in Machine Learning," *International Journal of Engineering Research*, vol. 4, Jan 2021.